UF CHIRPS v3.0 Zarr Pipeline
Technical Design Document (TDD)

1. Purpose

This Technical Design Document (TDD) describes **how** the University of Florida (UF) will implement the ingestion, validation, storage, and maintenance of **CHIRPS v3.0 daily precipitation data** using the **Zarr** format.

This document follows and implements the approved product specification and is intended for engineers, data infrastructure teams, and technical reviewers. It is **self-contained** and sufficient to begin implementation without further clarification.

2. System Overview

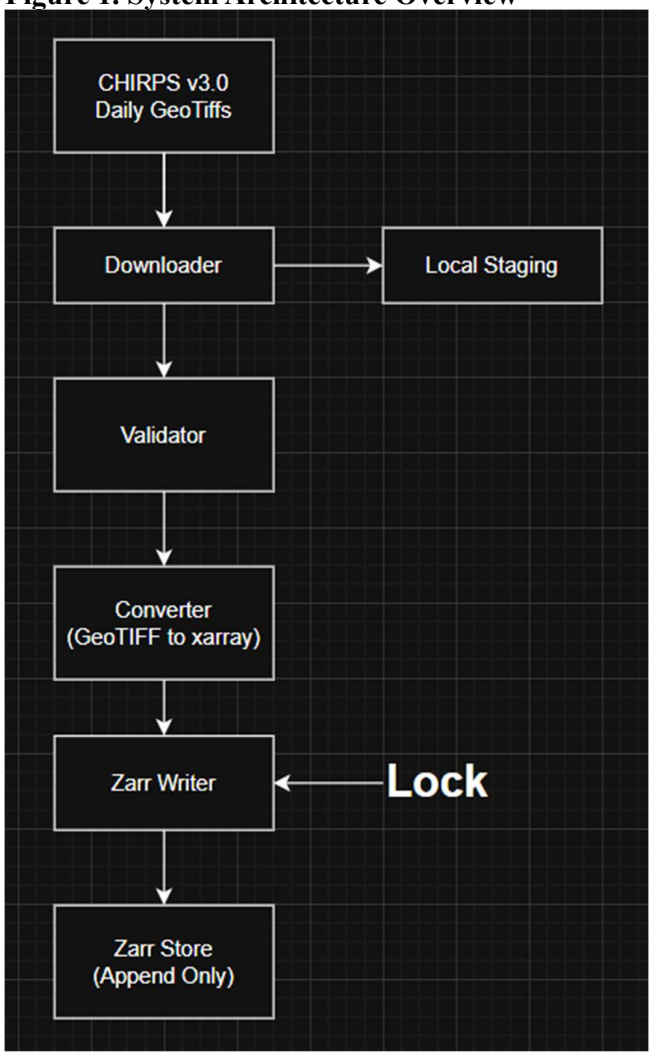
The system ingests daily CHIRPS v3.0 GeoTIFF files, validates them, converts them into a standardized xarray representation, and appends them to an immutable, versioned Zarr data store.

Key characteristics

- Single-writer, multi-reader model
- Append-only updates along the time dimension
- No modification of existing published data
- Separation of upstream scientific versioning and UF product versioning

3. High-Level Architecture

Figure 1. System Architecture Overview



This diagram illustrates the end-to-end data flow and enforces the single-writer rule.

Logical components:

1. **Ingestion Orchestrator** – controls daily workflow
2. **Downloader** – retrieves daily CHIRPS GeoTIFF files
3. **Validator** – enforces spatial, temporal, and format invariants
4. **Converter** – converts GeoTIFFs to xarray Datasets
5. **Zarr Writer** – appends validated data to the Zarr store
6. **Metadata Manager** – manages CF, provenance, and version metadata
7. **Logging & Audit Layer** – records ingestion outcomes

Each component has a single responsibility and can be tested independently.

4. Data Model

4.1 Dimensions

- **time**
  - Type: datetime64[ns]
  - Resolution: daily
  - Ordering: strictly increasing
  - Timezone: UTC
- **latitude**
  - Type: float32
  - Units: degrees_north
  - Representation: grid cell centers
- **longitude**
  - Type: float32
  - Units: degrees_east
  - Representation: grid cell centers

4.2 Data Variable

- **precipitation(time, latitude, longitude)**
  - Type: float32
  - Units: mm/day
  - Fill value: -9999.0
  - Valid range: ≥ 0

No additional data variables are included in this product.

5. Zarr Storage Design

5.1 Zarr Format

- Zarr format version: v2
- Storage backend: filesystem or object storage (abstracted)

5.2 Chunking

Chunks are fixed and immutable:

- time: 30 days
- latitude: ~250 cells
- longitude: ~250 cells

Chunking configuration must not change within a product version.

5.3 Compression

- Compressor: Blosc
- Codec: Zstandard (ZSTD)
- Compression level: moderate (e.g., level 3)
- Shuffle: bitshuffle

Compression is lossless and applied independently per chunk.

6. Directory and Naming Convention

6.1 Canonical Path Structure

uf-climate/chirps/v3.0/spatial/daily/precipitation/chirps_v3.0_daily_precip_v1.0.zarr

6.2 Version Semantics

- v3.0 denotes the upstream CHIRPS scientific version
- v1.0 denotes the UF product version

A new UF product version is created for any non-append change.

7. Bootstrap vs Incremental Ingestion

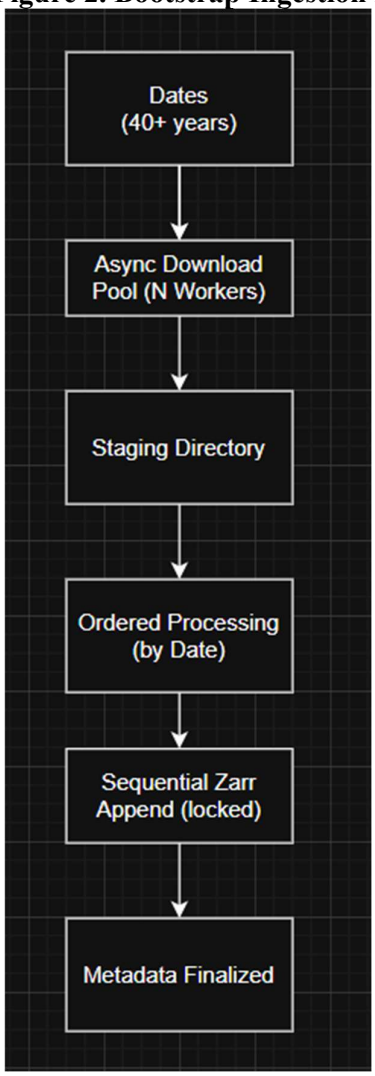
This system supports two distinct ingestion modes:

1. **Bootstrap Ingestion** – one-time historical backfill
2. **Incremental Ingestion** – daily append after bootstrap

Both modes use the same validation, conversion, and write logic, differing only in control flow.

7.1 Bootstrap Ingestion (Initial Zarr Creation)

Figure 2. Bootstrap Ingestion Flow (Historical Backfill)



Bootstrap Workflow

1. Verify that the target Zarr path does not already exist
2. Enumerate all available historical CHIRPS v3.0 daily GeoTIFFs in chronological order
3. Select the earliest date as time_coverage_start
4. Download the first GeoTIFF
5. Validate CRS, grid, and resolution
6. Initialize the Zarr store:
    - Create dimensions (time, latitude, longitude)
    - Write coordinate variables
    - Write dataset-level metadata
    - Write the first time slice
7. For each subsequent GeoTIFF:
    - Download
    - Validate spatial consistency
    - Convert to xarray
    - Append along the time dimension
8. After final day:
    - Set time_coverage_end
    - Persist final metadata
    - Mark dataset as **bootstrap complete**

Bootstrap ingestion is **restartable** and may be executed in batches.

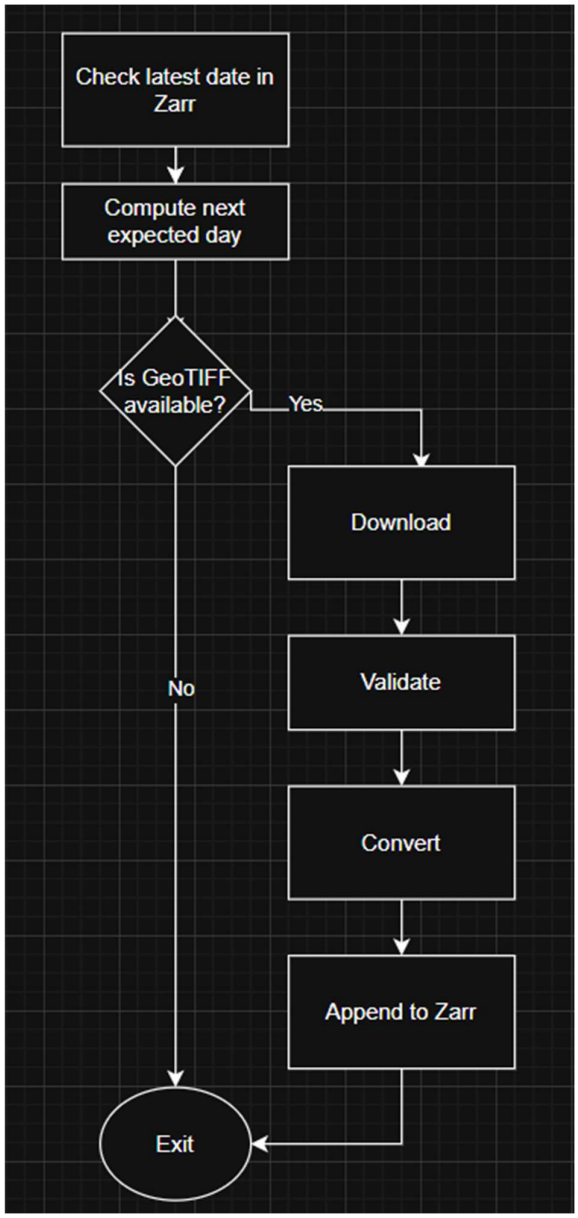
Parallelism is allowed only before the ordered processing stage.
Bootstrap ingestion is executed **only when no Zarr store exists**.

Purpose

- Create the initial Zarr dataset
- Populate it with the full historical CHIRPS v3.0 archive (~40+ years)
- Establish immutable dimensions, chunking, and metadata

7.2 Incremental Ingestion (Daily Updates)

Figure 3. Incremental Daily Ingestion Flow



8. Ingestion Workflow

Each ingestion cycle processes exactly one daily CHIRPS file.

Step-by-step flow

1. Determine the next expected date based on the latest time value in the Zarr store
2. Check availability of the corresponding CHIRPS v3.0 GeoTIFF
3. Download the GeoTIFF to a staging location
4. Validate file integrity (readable, non-empty)
5. Validate CRS (must be EPSG:4326)
6. Validate spatial grid consistency with existing dataset
7. Extract and validate date from filename
8. Convert GeoTIFF to xarray Dataset
9. Normalize dimensions and coordinates
10. Inject required metadata
11. Append data along the time dimension
12. Persist updated metadata
13. Log success or failure

If any validation step fails, ingestion for that date is aborted.

Incremental ingestion prioritizes simplicity and correctness over throughput.
Incremental ingestion begins only after successful bootstrap completion.

Purpose

- Append newly available daily CHIRPS data
- Preserve historical immutability

Incremental Workflow

1. Read the latest time value from the existing Zarr store
2. Compute the next expected date
3. Check availability of the corresponding CHIRPS GeoTIFF
4. If unavailable, exit without modification
5. If available, follow the standard validation and append process

7.3 Mode Selection Logic

At runtime, the system determines ingestion mode as follows:

- If Zarr store does not exist → **Bootstrap mode**
- If Zarr store exists → **Incremental mode**

Mode selection is automatic and requires no manual intervention.

8. Append and Idempotency Rules

- A date already present in the Zarr store must not be appended again
- Re-ingestion of an existing date results in a no-op or explicit failure
- Appends must not modify existing chunks

These rules ensure reproducibility and data integrity.

9. Validation Rules

9.1 Spatial Validation

- Latitude and longitude arrays must match exactly across all days
- Grid resolution must be 0.05°
- Coordinate ordering must be consistent

9.2 Temporal Validation

- One value per day
- No missing or duplicate timestamps
- Monotonic time ordering

9.3 Value Validation

- Precipitation values must be ≥ 0 or equal to fill value
- NaNs are converted to the defined fill value

10. Concurrency and Locking

10.1 Write Concurrency Model

- The Zarr store enforces a **single-writer** rule at all times
- Only one process may perform writes or appends
- Locking is mandatory during any Zarr write operation

Read operations may occur concurrently but must not observe partial writes.

10.2 Parallelism During Bootstrap Ingestion

To reduce bootstrap time for the ~40-year historical archive, **parallelism is permitted only in pre-write stages**.

Allowed Parallel Operations

- Asynchronous or multithreaded download of GeoTIFF files
- Parallel checksum and integrity validation
- Parallel conversion from GeoTIFF to in-memory xarray objects

These operations may be executed concurrently across multiple days.

Restricted Operations

- Zarr initialization
- Zarr appends
- Metadata updates

All write operations must remain **strictly sequential** to preserve data integrity.

10.3 Bootstrap Execution Strategy

Recommended execution model:

1. Maintain an asynchronous download queue for historical dates
2. Download and stage files in parallel batches
3. Process staged files in strict chronological order
4. Append one day at a time to the Zarr store under an exclusive lock

This approach minimizes idle time while preserving append guarantees.

10.4 Incremental Ingestion Parallelism

Incremental (daily) ingestion does not require parallelism:

- Only one file is processed per run
- Simplicity and reliability are prioritized over throughput

11. Failure Handling and Recovery

- Only one ingestion process may write to a Zarr store at a time

- File-based or storage-native locking is required during writes
- Reads are permitted during ingestion but must not observe partial writes

11. Failure Handling and Recovery
- Failed ingestions do not modify the Zarr store
- Temporary files are cleaned up on failure
- Failures are logged with date, reason, and stack trace
- Retrying ingestion for a failed date is permitted

12. Metadata Management
12.1 Standards
- CF Metadata Conventions: CF-1.9

12.2 Metadata Categories
- Scientific metadata (variables, units, coordinates)
- Provenance metadata (source, retrieval date, pipeline version)
- Operational metadata (institution, update frequency)

Metadata updates must be deterministic and auditable.

13. Logging and Auditing
The system records:
- Ingestion date
- Target dataset version
- Success or failure
- Validation errors (if any)

Logs must be retained for audit and troubleshooting purposes.

14. Acceptance Criteria
The system is considered correct when:
- Daily data can be appended without modifying existing data
- All validation rules are enforced
- Metadata complies with CF-1.9
- Zarr store is readable using standard xarray tools

15. Explicit Non-Goals
The following are out of scope for this design:
- Derived products (monthly, climatology, anomalies)
- API or web service implementation
- Performance benchmarking
- User authentication or access control
- Scientific modification of CHIRPS data

16. Summary
This Technical Design Document defines a robust, append-safe, and standards-compliant implementation for hosting CHIRPS v3.0 daily precipitation data at the University of Florida using Zarr. It provides clear guidance for implementation while preserving scientific integrity, reproducibility, and long-term maintainability.